Project Proposal
CS24S009

Parallel Ray-Tracing Algorithm for Rendering Analytically Defined Objects with Point Classification

Motivation

Ray-tracing produces some of the most realistic computer-generated images we see today in movies, architectural walkthroughs, and scientific simulations. It works by simulating how light actually travels: we shoot rays from a virtual camera through every pixel, find what each ray hits, and compute the color based on how light interacts with the surface. The result is naturally accurate shadows, reflections, and refractions, things that older methods like rasterization can only approximate.

The trouble is that ray-tracing is *slow*. A single high-resolution image might need millions of rays, each tested against every object in the scene. Each ray can bounce multiple times, spawning more rays at every bounce. This makes parallel computing not just helpful but essential.

Our project aims to develop a parallel ray-tracing algorithm from scratch. We will render analytically defined objects like spheres, planes, cylinders, and shapes built from Boolean combinations of these (Constructive Solid Geometry). We will also implement a module that classifies any given 3D point as lying *inside*, *on*, or *outside* a given object, which is important for CSG evaluation and refraction calculations.

Why This Problem is Naturally Parallel

This is your text on the left side, and it will wrap around the image. Keep writing more text here to see the effect.

Ray-tracing is what parallel computing textbooks call an "embarrassingly parallel" problem. Each pixel is independent i.e. the ray through pixel $(100, 200)$ has nothing to do with the ray through pixel $(300, 400)$. We can divide the image among hundreds of processors and they can all work simultaneously without waiting for each other.

However, it is not *perfectly* balanced in practice. A ray hitting a mirror spawns reflection rays; a ray aimed at empty sky terminates instantly. Some image regions are far more expensive than others. This makes load balancing a real challenge and an area where novel contributions are still possible.

The Cornell Box as Our Validation Standard

The Cornell Box was created in 1984 at Cornell University by Goral, Torrance, Greenberg, and Battaile for their SIGGRAPH '84 paper. It is a simple room: red left wall, green right wall, white floor, ceiling, and back wall, an area light on the ceiling, and two white blocks inside. A physical version was built and photographed with calibrated equipment so that rendered images could be compared against the real thing.

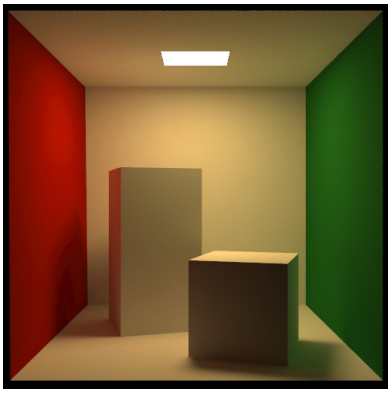


Figure 1: The Cornell Box

What makes it special is the *color bleeding* effect: light bouncing off the red wall tints nearby white surfaces pink, and similarly for the green wall. This effect is impossible without proper global illumination, so if our renderer shows it, we know the light transport is working. We will use the Cornell Box as our primary test scene and compare output against reference renderers like PBRT and Mitsuba 3.

Our Approach: Hybrid OpenMP + MPI Parallelization

We will implement the ray-tracer in C/C++ using a hybrid approach. **OpenMP** handles parallelism within each compute node, distributing pixel work across CPU cores with dynamic scheduling to manage load imbalance. **MPI** distributes work across multiple cluster nodes, each rank renders a set of image tiles independently, with the final image gathered at the root process. We will explore several directions for novel contributions:

- **NUMA-aware BVH traversal:** On multi-socket systems, memory access speed depends on which socket owns the data. We will partition the acceleration structure across NUMA domains and schedule rays to threads with fast local access to the relevant scene data.
- **Adaptive load balancing:** Instead of equal-sized tiles, we will use cost estimation from preview renders to assign smaller tiles to expensive regions, achieving better utilization across MPI ranks.

These areas are notably under-explored in current literature, which is heavily GPU-focused. A rigorous CPU-cluster study with novel load balancing and NUMA-awareness has strong publication potential in venues such as EGPGV, JPDC, or Parallel Computing.